DEPARTEMEN AKUNTANSI · FAKULTAS EKONOMIKA DAN BISNIS
UNIVERSITAS DIPONEGORO

Lembar Jawaban Ujian Tengah Semester

Pemrograman / Coding · Semester Gasal 2026/2027

Nama	**Zayyan Beryl**
NIM	**12030124140283**
Mata kuliah / Kelas	**Pengkodean dan Pemrograman · Kelas E**
Dosen pengampu	**Totok Dewayanto**

Repositori Git (seluruh artefak jawaban)
https://github.com/zayyanberyl-coding/uts-coding-akuntansi

Purwarupa web Voucher Kas Keluar (Soal 3)
https://zayyanberyl-coding.github.io/uts-coding-akuntansi/soal3/web/

Basis data cloud
Supabase (PostgreSQL) · project `uts-coding-akuntansi`

Alat bantu yang digunakan (ujian bersifat Open Book / Open AI / Open Code Sandbox): PostgreSQL 16 (uji lokal), Supabase, Gemini Canvas, NotebookLM, serta asisten AI Claude untuk membantu penyusunan dan pengujian kode. Seluruh kode diuji dan hasil ujinya dilampirkan di folder `bukti/` pada repositori.

Daftar Isi

1. Soal 1: Desain Arsitektur Data, Normalisasi & Asas Berpasangan (25%)
2. Soal 2: Otomasi Bisnis, Stored Procedure & Trigger Persediaan (40%)
3. Soal 3: Purwarupa Web Full-Stack & Orkestrasi AI Tools (35%)
4. Lampiran: Struktur repositori, cara menjalankan, dan bukti uji

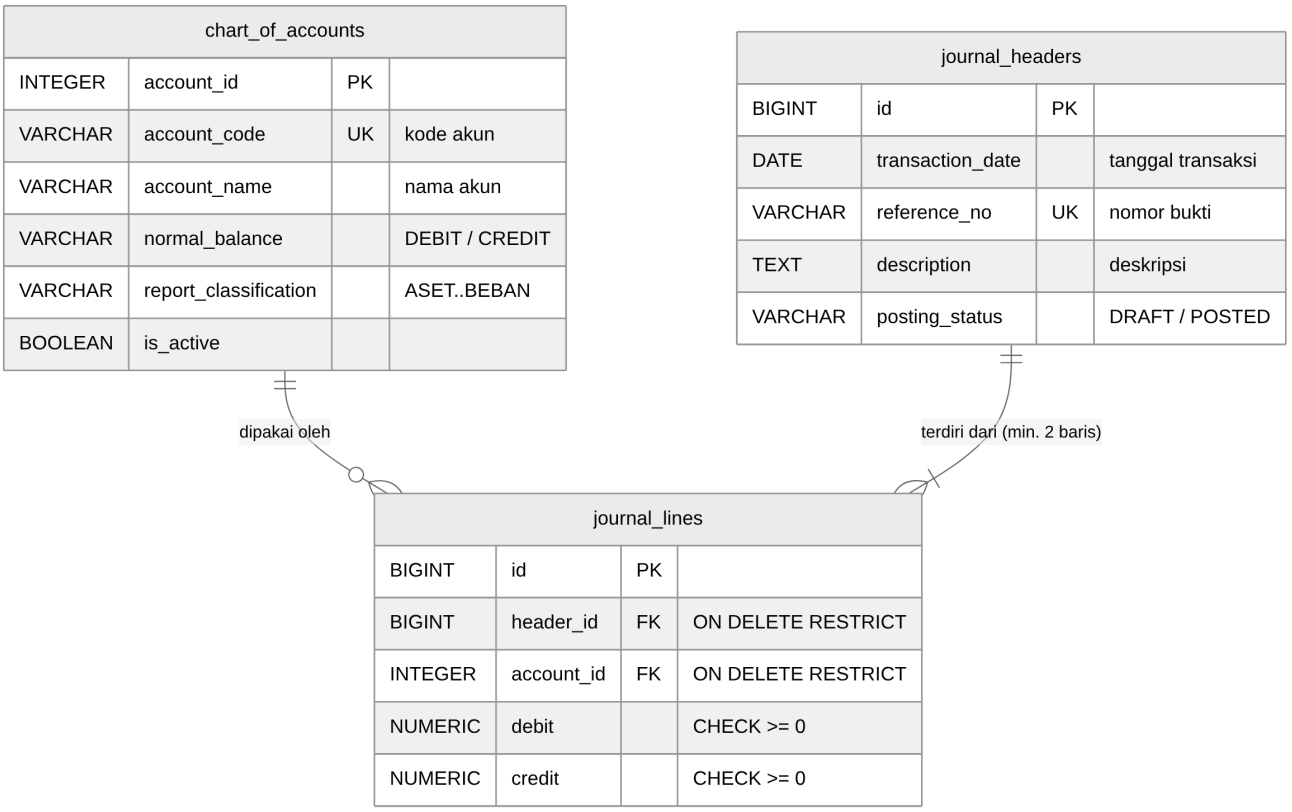
Ringkasan

Soal	Artefak utama	Bukti uji
1	ERD, DDL 3 tabel (PK, FK ON DELETE RESTRICT, CHECK) + constraint trigger Debit = Kredit	8 skenario: 1 diterima, 7 pelanggaran ditolak
2	`trg_update_stock_and_journal()` + jurnal HPP/Persediaan otomatis; `SELECT … FOR UPDATE`, SERIALIZABLE	6 skenario + rekonsiliasi GL = stok fisik; simulasi 2 kasir bersamaan
3	Formulir web (HTML+Tailwind+JS) + Supabase RPC; immutability, storno, created_by/created_at/hash SHA-256 berantai, audit log	15 skenario + uji keamanan anon + tindak lanjut NotebookLM

Soal 1: Desain Arsitektur Data, Normalisasi & Penegakan Asas Berpasangan

File terkait: `soal1/01_schema_general_ledger.sql`, `soal1/02_uji_soal1.sql`, `soal1/erd.mmd`, `soal1/erd.png`, `bukti/uji_soal1.txt`

1.1 Perancangan Skema Relasional (ERD)



Relasi antar entitas:

Relasi	Kardinalitas	Arti
`journal_headers` → `journal_lines`	1 : N (minimal 2)	Satu bukti transaksi terdiri dari beberapa baris debit/kredit. Jurnal yang di-POSTED wajib punya minimal 2 baris (satu sisi debit, satu sisi kredit).
`chart_of_accounts` → `journal_lines`	1 : N (boleh 0)	Satu akun bisa dipakai oleh banyak baris jurnal. Akun yang belum pernah dipakai tetap sah.

`journal_lines` adalah **tabel asosiatif**, artinya tabel penghubung antara bukti transaksi dan akun. Setiap barisnya menjawab pertanyaan: "pada bukti X, akun Y didebit/dikredit sebesar berapa?"

Keputusan desain yang penting:

- `account_code` dan `reference_no` diberi **UNIQUE**, sehingga kode akun dan nomor bukti tidak bisa ganda.
- `normal_balance` disimpan per akun karena ada **akun kontra**. Contohnya Akumulasi Penyusutan termasuk ASET tetapi saldo normalnya KREDIT. Jadi saldo normal tidak bisa ditebak hanya dari kelompok akunnya.

- Jenis laporan (Neraca/Laba Rugi) **tidak disimpan** sebagai kolom karena bisa diturunkan dari `report_classification` lewat view `v_chart_of_accounts`. Kalau disimpan, akan muncul ketergantungan transitif yang melanggar 3NF.

1.2 Kekangan Integritas (DDL PostgreSQL)

DDL lengkap ada di `soal1/01_schema_general_ledger.sql`. Inti tabel `journal_lines`:

```
CREATE TABLE journal_lines (
    id         BIGINT GENERATED ALWAYS AS IDENTITY,
    header_id  BIGINT        NOT NULL,
    account_id INTEGER       NOT NULL,
    debit      NUMERIC(18,2) NOT NULL DEFAULT 0,
    credit     NUMERIC(18,2) NOT NULL DEFAULT 0,
    CONSTRAINT pk_journal_lines PRIMARY KEY (id),
    CONSTRAINT fk_lines_header  FOREIGN KEY (header_id)
        REFERENCES journal_headers (id) ON DELETE RESTRICT,
    CONSTRAINT fk_lines_account FOREIGN KEY (account_id)
        REFERENCES chart_of_accounts (account_id) ON DELETE RESTRICT,
    CONSTRAINT ck_lines_debit_nonneg  CHECK (debit  >= 0),
    CONSTRAINT ck_lines_credit_nonneg CHECK (credit >= 0),
    CONSTRAINT ck_lines_not_both_zero CHECK (NOT (debit = 0 AND credit = 0)),
    CONSTRAINT ck_lines_one_side      CHECK (NOT (debit > 0 AND credit > 0))
);
```

Nilai tambah: keseimbangan Debit = Kredit ditegakkan di database. CHECK constraint hanya bisa melihat satu baris, padahal keseimbangan jurnal melibatkan banyak baris. Karena itu dipakai `CONSTRAINT TRIGGER ... DEFERRABLE INITIALLY DEFERRED` yang menghitung `SUM(debit)` dan `SUM(credit)` per jurnal **saat COMMIT**. Ini langsung menutup celah pada kasus soal: transaksi yang dikirim lewat API tanpa melalui frontend tetap ditolak database. Selain itu, header wajib dibuat sebagai `DRAFT` dan hanya bisa diubah ke `POSTED` bila punya minimal 2 baris yang seimbang.

Hasil uji (`bukti/uji_soal1.txt`, dijalankan di PostgreSQL 16):

No	Skenario	Hasil
1	Jurnal seimbang 500.000 = 500.000	✅ Tersimpan & POSTED
2	Jurnal tidak seimbang (750.000 vs 700.000) dikirim langsung	❌ Ditolak saat COMMIT: *"Jurnal tidak seimbang"*
3	Debit negatif	❌ Ditolak `ck_lines_debit_nonneg`
4	Debit & kredit sama-sama 0	❌ Ditolak `ck_lines_not_both_zero`
5	Hapus akun Kas yang sudah dipakai	❌ Ditolak `fk_lines_account` (RESTRICT)
6	Hapus header yang punya baris	❌ Ditolak `fk_lines_header` (RESTRICT)
7	Posting jurnal tanpa baris	❌ Ditolak aturan posting
8	Akun yang tidak ada di bagan akun	❌ Ditolak foreign key

1.3 Analisis Audit & Keamanan Finansial: Header-Line vs Tabel Tunggal

Model tabel tunggal (flat), misalnya `jurnal(no_bukti, tanggal, deskripsi, akun_debit, akun_kredit, nominal)` atau satu baris per akun yang mengulang tanggal dan deskripsi, punya kelemahan berikut dibandingkan model Header-Line.

a. Dari sisi normalisasi (3NF)

- Pada tabel flat, `tanggal`, `deskripsi`, dan `status` bergantung pada `no_bukti`, bukan pada kunci baris jurnal. Ini **ketergantungan parsial/transitif** yang melanggar 2NF/3NF, karena atribut header diulang di setiap baris.
- Pengulangan data memicu **anomali**:
  - *Anomali update*: tanggal diubah di satu baris tetapi tidak di baris lain, sehingga satu bukti punya dua tanggal.
  - *Anomali insert*: bukti tidak bisa dicatat tanpa baris akun.
  - *Anomali delete*: menghapus satu baris akun bisa membuat sisa jurnal tidak seimbang tanpa terdeteksi.
- Pada model Header-Line, atribut bukti disimpan **sekali** di `journal_headers`, sedangkan `journal_lines` hanya berisi atribut yang bergantung penuh pada barisnya (akun, debit, kredit). Model ini memenuhi 3NF.

b. Dari sisi jejak audit (audit trail)

- **Satu bukti = satu identitas.** Auditor bisa menelusuri dari laporan keuangan → buku besar → baris jurnal → header → nomor bukti fisik (vouching), lalu sebaliknya (tracing). Nomor bukti yang `UNIQUE` mencegah bukti ganda dan memudahkan uji kelengkapan urutan nomor.
- **Keseimbangan bisa diuji per transaksi.** `SUM(debit) = SUM(credit) GROUP BY header_id` bisa ditegakkan otomatis (constraint trigger di atas). Pada tabel flat dengan pola "akun_debit/akun_kredit/nominal", jurnal majemuk (satu debit, banyak kredit) harus dipecah sehingga keterkaitannya hilang. Pada pola satu-baris-per-akun tanpa header, tidak ada kunci yang andal untuk mengelompokkan baris.
- **Status dan otorisasi melekat pada header.** Status `DRAFT/POSTED`, pembuat, waktu, dan checksum (Soal 3.2) cukup dicatat sekali di header dan berlaku untuk seluruh barisnya. Penguncian jurnal yang sudah POSTED dan koreksi lewat jurnal pembalik menjadi sederhana dan konsisten.
- **Integritas referensial.** `ON DELETE RESTRICT` mencegah akun atau bukti terhapus selama masih punya riwayat transaksi, sehingga tidak ada "baris yatim" yang menghilangkan jejak.
- **Pengendalian internal.** Validasi tidak bergantung pada frontend. Seluruh aturan (CHECK, FK, keseimbangan, aturan posting) berada di lapisan basis data yang dipakai bersama oleh semua kanal (web, API, impor), sesuai prinsip *defense in depth*.

Kesimpulan: model Header-Line lebih andal karena menghilangkan redundansi (3NF), memungkinkan pengujian keseimbangan per bukti, dan memberi titik tunggal untuk status, otorisasi, dan jejak audit. Model ini mencegah kasus *unbalanced balance sheet* yang dialami sistem lama.

Cuplikan bukti uji Soal 1 (PostgreSQL 16)

```
=== UJI 2: Jurnal TIDAK seimbang dikirim langsung (simulasi API) -> HARUS DITOLAK saat COMMIT
psql:soal1/02_uji_soal1.sql:29: ERROR:  Jurnal 2 tidak seimbang: 2 baris, Total Debit = 750000.00, Total
Kredit = 700000.00
=== UJI 3: Nominal debit negatif -> HARUS DITOLAK (ck_lines_debit_nonneg)
psql:soal1/02_uji_soal1.sql:33: ERROR:  new row for relation "journal_lines" violates check constraint
"ck_lines_debit_nonneg"
=== UJI 4: Debit dan kredit sama-sama nol -> HARUS DITOLAK (ck_lines_not_both_zero)
psql:soal1/02_uji_soal1.sql:37: ERROR:  new row for relation "journal_lines" violates check constraint
"ck_lines_not_both_zero"
=== UJI 5: Hapus akun Kas yang sudah dipakai jurnal -> HARUS DITOLAK (ON DELETE RESTRICT)
psql:soal1/02_uji_soal1.sql:40: ERROR:  update or delete on table "chart_of_accounts" violates foreign key
constraint "fk_lines_account" on table "journal_lines"
=== UJI 6: Hapus header jurnal yang punya baris -> HARUS DITOLAK (ON DELETE RESTRICT)
psql:soal1/02_uji_soal1.sql:43: ERROR:  update or delete on table "journal_headers" violates foreign key
constraint "fk_lines_header" on table "journal_lines"
=== UJI 7: Posting jurnal yang baru punya 0 baris -> HARUS DITOLAK
psql:soal1/02_uji_soal1.sql:49: ERROR:  Jurnal 3 tidak dapat di-POSTED: minimal 2 baris (debit & kredit).
=== UJI 8: Akun tidak terdaftar di bagan akun -> HARUS DITOLAK (FOREIGN KEY)
psql:soal1/02_uji_soal1.sql:54: ERROR:  insert or update on table "journal_lines" violates foreign key
constraint "fk_lines_account"
```

```
=== HASIL AKHIR: hanya jurnal JU-0001 yang tersimpan, dan seimbang
 reference_no | posting_status | account_code | account_name |   debit   |  credit
--------------+----------------+--------------+--------------+-----------+-----------
 JU-0001      | POSTED         | 6103         | Beban Sewa   | 500000.00 |      0.00
 JU-0001      | POSTED         | 1101         | Kas          |      0.00 | 500000.00
(2 rows)

 reference_no | total_debit | total_kredit
--------------+-------------+--------------
 JU-0001      |   500000.00 |    500000.00
(1 row)
```

Soal 2: Otomasi Bisnis, Stored Procedure & Database Trigger Persediaan

File terkait: `soal2/01_inventory_trigger.sql`, `soal2/02_uji_trigger.sql`, `soal2/03_concurrency_snippets.sql`, `soal2/concurrency_demo.sh`, `bukti/uji_soal2_trigger.txt`, `bukti/uji_soal2_concurrency.txt`

2.1 Otomatisasi Mutasi via Database Trigger

Cara kerja `trg_update_stock_and_journal()`

Trigger dipasang `BEFORE INSERT ... FOR EACH ROW` pada `inventory_transactions`. Untuk setiap mutasi, langkahnya:

1. **Validasi qty.** `PURCHASE` dan `SALE` wajib > 0. `ADJUSTMENT` boleh bertanda (+ untuk barang masuk atau retur, – untuk rusak atau hilang) tetapi tidak boleh 0. Ini mencegah trik "SALE qty negatif" untuk menambah stok.
2. **Mengunci baris item** dengan `SELECT ... FOR UPDATE`. Transaksi lain pada item yang sama harus antre (lihat 2.2).
3. **Menghitung saldo baru**: `stock_qty + delta` (PURCHASE +, SALE −, ADJUSTMENT ±).
4. **Anti-negative inventory**: bila saldo baru < 0, dijalankan `RAISE EXCEPTION`. Seluruh transaksi database **dibatalkan** (rollback), termasuk baris mutasi dan jurnalnya, sehingga tidak ada data setengah jadi.
5. **Memperbarui `inventory_items.stock_qty`.**
6. **Memutakhirkan buku besar secara otomatis.** Ini menjawab akar masalah PT Diponegoro Logistik Utama (saldo fisik berubah tetapi akun Persediaan/HPP terlambat diperbarui). Jurnal dibuat dalam transaksi yang **sama** dengan mutasi stok:

Jenis mutasi	Debit	Kredit
PURCHASE	1301 Persediaan Barang Dagang	2101 Utang Usaha
SALE	5101 Beban Pokok Pendapatan (HPP)	1301 Persediaan Barang Dagang
ADJUSTMENT (+)	1301 Persediaan Barang Dagang	4901 Pendapatan Selisih Persediaan
ADJUSTMENT (−)	5901 Beban Selisih Persediaan	1301 Persediaan Barang Dagang

Nilai jurnal = qty × `unit_cost`. Nomor jurnal `INV-<trx_id>` disimpan di kolom `journal_header_id`, sehingga setiap mutasi bisa ditelusuri ke jurnalnya dan sebaliknya.

Kode inti:

```
CREATE OR REPLACE FUNCTION trg_update_stock_and_journal()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    v_item inventory_items%ROWTYPE;  v_delta INT;  v_new_stock INT;
BEGIN
    SELECT * INTO v_item FROM inventory_items
     WHERE item_id = NEW.item_id FOR UPDATE;            -- kunci baris

    v_delta := CASE NEW.trx_type WHEN 'PURCHASE' THEN NEW.qty
                                 WHEN 'SALE'     THEN -NEW.qty
                                 ELSE NEW.qty END;
    v_new_stock := v_item.stock_qty + v_delta;

    IF v_new_stock < 0 THEN                             -- (b) anti-negative
        RAISE EXCEPTION 'Transaksi % ditolak: stok % hanya % unit, diminta % unit.',
              NEW.trx_type, v_item.item_id, v_item.stock_qty, ABS(v_delta);
    END IF;
```

```
    UPDATE inventory_items SET stock_qty = v_new_stock    -- (a) update stok
     WHERE item_id = NEW.item_id;

    -- (+) jurnal Persediaan/HPP otomatis -> NEW.journal_header_id
    ...
    RETURN NEW;
END; $$;

CREATE TRIGGER trg_update_stock_and_journal
    BEFORE INSERT ON inventory_transactions
    FOR EACH ROW EXECUTE FUNCTION trg_update_stock_and_journal();
```

Sebagai lapis pengaman terakhir ditambahkan juga `CHECK (stock_qty >= 0)` pada `inventory_items`, sehingga stok tidak bisa negatif walaupun ada kode lain yang meng-UPDATE stok tanpa melalui trigger.

Hasil uji (`bukti/uji_soal2_trigger.txt`)

No	Skenario	Stok	Hasil
1	PURCHASE 10 Palet Kayu @150.000	0 → 10	✅ Jurnal Dr Persediaan 1.500.000 / Cr Utang Usaha
2	SALE 4	10 → 6	✅ Jurnal Dr HPP 600.000 / Cr Persediaan
3	ADJUSTMENT −1 (rusak saat opname)	6 → 5	✅ Jurnal Dr Beban Selisih 150.000 / Cr Persediaan
4	SALE 6 padahal stok 5	tetap 5	❌ `RAISE EXCEPTION` stok tidak boleh negatif
5	SALE qty −3	tetap 5	❌ Qty harus > 0
6	Dua SALE (3 + 3) dalam satu transaksi	tetap 5	❌ SALE kedua gagal, **SALE pertama ikut dibatalkan**

Rekonsiliasi: saldo akun 1301 Persediaan di buku besar = Rp750.000 = nilai stok fisik (5 × Rp150.000). Tidak ada selisih saat stock opname.

2.2 Penanganan Concurrency (Race Condition)

Masalahnya

Misalnya stok tinggal 1 unit. Kasir A dan Kasir B sama-sama membaca stok = 1, sama-sama lolos pengecekan "stok ≥ 1", lalu sama-sama menulis stok = 0. Hasilnya **2 unit terjual padahal stok hanya 1**. Ini disebut *lost update* yang menimbulkan *overselling / phantom stock*.

Teknik yang diterapkan

1. Pessimistic locking: `SELECT ... FOR UPDATE` (dipakai di dalam trigger). Baris item dikunci sejak dibaca. Kasir B tidak bisa membaca-untuk-mengubah baris yang sama sampai Kasir A selesai (COMMIT/ROLLBACK). Setelah itu Kasir B membaca stok **terbaru** (0) dan ditolak oleh `RAISE EXCEPTION`. Isolasi default `READ COMMITTED` sudah cukup karena penguncian bersifat eksplisit.

```
BEGIN;
  SELECT stock_qty FROM inventory_items
   WHERE item_id = 'BRG-002'
     FOR UPDATE;                    -- kasir lain antre di sini
  INSERT INTO inventory_transactions (item_id, trx_type, qty)
  VALUES ('BRG-002', 'SALE', 1);     -- trigger: cek stok & update
COMMIT;
-- opsional: FOR UPDATE NOWAIT  atau  SET lock_timeout = '3s';
```

2. Isolasi `SERIALIZABLE` + retry. PostgreSQL mendeteksi bahwa dua transaksi saling bergantung, lalu membatalkan salah satunya dengan `ERROR 40001 could not serialize access`. Aplikasi **wajib mengulang** (retry) transaksi yang gagal.

```
BEGIN ISOLATION LEVEL SERIALIZABLE;
  SELECT stock_qty FROM inventory_items WHERE item_id = 'BRG-002';
  UPDATE inventory_items SET stock_qty = stock_qty - 1 WHERE item_id = 'BRG-002';
COMMIT;   -- transaksi yang kalah: SQLSTATE 40001 -> retry
```

Contoh retry di sisi aplikasi (Node.js) ada di `soal2/03_concurrency_snippets.sql`.

3. Alternatif ringkas: atomic conditional update. `UPDATE ... SET stock_qty = stock_qty - 1 WHERE item_id = $1 AND stock_qty >= 1 RETURNING stock_qty;` Pengecekan dan pengurangan terjadi dalam satu perintah. Bila tidak ada baris yang kembali, stok habis.

4. Lapis terakhir: `CHECK (stock_qty >= 0)`.

Bukti simulasi dua kasir (`soal2/concurrency_demo.sh` → `bukti/uji_soal2_concurrency.txt`)

Demo	Teknik	Hasil
A	Tanpa penguncian (baca-cek-tulis di aplikasi)	❌ Kedua kasir berhasil: **2 unit terjual, stok 1** (overselling)
B	`SELECT ... FOR UPDATE` di trigger	✅ Kasir B mulai 0,5 detik setelah A, **tertahan ±2,5 detik** sampai A commit, lalu ditolak "stok hanya 0 unit". Hanya 1 penjualan tersimpan.
C	`SERIALIZABLE`	✅ Kasir B mendapat `ERROR 40001`, hanya 1 unit terjual

Rekomendasi: gunakan `FOR UPDATE` di dalam trigger (sudah diterapkan) karena sederhana, deterministik, dan tidak memerlukan logika retry. `SERIALIZABLE` cocok untuk transaksi kompleks yang membaca banyak baris, asalkan aplikasi menyediakan mekanisme retry.

Cuplikan bukti: trigger & rekonsiliasi

```
=== UJI 4: SALE 6 unit padahal stok 5 -> HARUS DITOLAK (RAISE EXCEPTION), stok tetap 5
psql:soal2/02_uji_trigger.sql:17: ERROR:  Transaksi SALE ditolak: stok BRG-001 (Palet Kayu Standar) hanya
5 unit, diminta 6 unit. Stok tidak boleh negatif.
HINT:  Periksa stok fisik atau catat PURCHASE/ADJUSTMENT terlebih dahulu.
CONTEXT:  PL/pgSQL function trg_update_stock_and_journal() line 39 at RAISE
=== UJI 5: SALE qty negatif (trik mengakali stok) -> HARUS DITOLAK
psql:soal2/02_uji_trigger.sql:20: ERROR:  Qty -3 untuk transaksi SALE harus lebih dari 0
CONTEXT:  PL/pgSQL function trg_update_stock_and_journal() line 13 at RAISE
=== UJI 6: Rollback menyeluruh - 2 penjualan dalam 1 transaksi, yang kedua gagal
psql:soal2/02_uji_trigger.sql:25: ERROR:  Transaksi SALE ditolak: stok BRG-001 (Palet Kayu Standar) hanya
2 unit, diminta 3 unit. Stok tidak boleh negatif.
HINT:  Periksa stok fisik atau catat PURCHASE/ADJUSTMENT terlebih dahulu.
CONTEXT:  PL/pgSQL function trg_update_stock_and_journal() line 39 at RAISE
    -> stok harus tetap 5 (penjualan pertama ikut dibatalkan)
=== POSISI STOK
 item_id |      item_name      | stock_qty | unit_cost | nilai_persediaan
---------+---------------------+-----------+-----------+------------------
 BRG-001 | Palet Kayu Standar  |         5 | 150000.00 |        750000.00
 BRG-002 | Stretch Film 50cm   |         0 |  85000.00 |             0.00
 BRG-003 | Kardus Ekspor Besar |         0 |  12500.00 |             0.00
(3 rows)
```

```
=== JURNAL OTOMATIS DI BUKU BESAR
 reference_no | account_code |          account_name            |   debit    |   credit
--------------+--------------+----------------------------------+------------+------------
 INV-000001   | 1301         | Persediaan Barang Dagang         | 1500000.00 |       0.00
 INV-000001   | 2101         | Utang Usaha                      |       0.00 | 1500000.00
 INV-000002   | 5101         | Beban Pokok Pendapatan (HPP)     |  600000.00 |       0.00
 INV-000002   | 1301         | Persediaan Barang Dagang         |       0.00 |  600000.00
 INV-000003   | 5901         | Beban Selisih/Kerugian Persediaan |  150000.00 |       0.00
 INV-000003   | 1301         | Persediaan Barang Dagang         |       0.00 |  150000.00
(6 rows)
```

```
=== REKONSILIASI: saldo akun Persediaan (GL) = nilai stok fisik?
 saldo_gl_persediaan | nilai_stok_fisik
---------------------+------------------
          750000.00 |        750000.00
(1 row)
```

Cuplikan bukti: simulasi dua kasir

```
# DEMO A - TANPA PENGUNCIAN (pola baca-cek-tulis di aplikasi)
################################################################
[Kasir A] membaca stok = 1
[Kasir B] membaca stok = 1
[Kasir A] PENJUALAN BERHASIL
[Kasir B] PENJUALAN BERHASIL
 stok_akhir | unit_terjual
------------+--------------
          0 |            2
(1 row)

>> MASALAH: 2 unit terjual padahal stok hanya 1 (overselling / phantom stock).

################################################################
# DEMO B - SELECT ... FOR UPDATE (di dalam trg_update_stock_and_journal)
################################################################
 item_id |     item_name     | stok_awal
---------+-------------------+-----------
 BRG-002 | Stretch Film 50cm |         1
(1 row)

[Kasir B] 20:12:40.303 BEGIN + INSERT SALE 1 unit BRG-002
[Kasir B] ERROR:  Transaksi SALE ditolak: stok BRG-002 (Stretch Film 50cm) hanya 0 unit, diminta 1 unit.
Stok tidak boleh negatif.
[Kasir A] 20:12:39.801 BEGIN + INSERT SALE 1 unit BRG-002
[Kasir A] 20:12:39.808 INSERT lolos, baris item TERKUNCI (FOR UPDATE). Proses pembayaran 3 detik...
[Kasir A] 20:12:42.814 COMMIT - PENJUALAN BERHASIL
[Kasir B] 20:12:42.821 transaksi dibatalkan (ROLLBACK) setelah menunggu kunci Kasir A
 stok_akhir
------------
          0
(1 row)

 trx_id | trx_type | qty
--------+----------+-----
      8 | PURCHASE |   1
      9 | SALE     |   1
(2 rows)

>> HASIL: hanya 1 penjualan tersimpan, Kasir B ditolak setelah Kasir A commit. Stok = 0, tidak negatif.

################################################################
# DEMO C - ISOLASI SERIALIZABLE
################################################################
[Kasir A] (SERIALIZABLE) membaca stok = 1
[Kasir A] COMMIT - PENJUALAN BERHASIL
[Kasir B] (SERIALIZABLE) membaca stok = 1
[Kasir B] ERROR:  could not serialize access due to concurrent update
 stok_akhir | unit_terjual
------------+--------------
          0 |            1
(1 row)

>> HASIL: salah satu kasir mendapat ERROR 40001 (serialization failure) dan harus retry; tidak ada
overselling.
```

Soal 3: Purwarupa Web Full-Stack & Orkestrasi AI Tools

Purwarupa online: https://zayyanberyl-coding.github.io/uts-coding-akuntansi/soal3/web/ (tambahkan `?demo` untuk mencoba tanpa login) **Basis data cloud:** Supabase (PostgreSQL), project `uts-coding-akuntansi`

Arsitektur:

```
Browser (HTML + Tailwind + JS)  --supabase.rpc()-->  Supabase PostgREST  -->  PostgreSQL
  - live balanced indicator                            (JWT pengguna)          - RPC SECURITY DEFINER
  - checklist aturan bisnis                                                     - trigger immutability
  - Submit terkunci s.d. valid                                                  - hash SHA-256 berantai
                                                                                - audit_log append-only
```

3.1 Orkestrasi AI Tools

a. Gemini Canvas: formulir entri jurnal dinamis

Prompt lengkap, riwayat percobaan, dan evaluasi kode ada di `soal3/prompts/01_gemini_canvas_prompt.md`.

- **Prompt** disusun dengan pola *peran → batasan teknis → aturan bisnis bernomor → kontrak payload → desain*. Tujuh aturan bisnis ditulis sebagai kriteria yang dapat diuji (total otomatis, indikator SEIMBANG/TIDAK SEIMBANG + selisih, nominal non-negatif & satu sisi, kas hanya di kredit, tanggal tidak di masa depan, tombol Submit terkunci + checklist, perhitungan dalam sen).
- **Hasil Gemini** (Gemini 3.6 Flash + Canvas, setelah dua percobaan dengan Gemini Pro gagal): formulir lengkap dengan indikator **SEIMBANG (BALANCED)**, checklist 5 aturan, dan payload JSON.
- **Code review** menemukan bug yang halus: konversi sen tanpa pembulatan (`0.29*100 = 28.999…`), "hari ini" dihitung dalam UTC (salah untuk WIB 00.00–07.00), sen terpotong di payload, dan validasi hanya di browser.
- **Versi final** (`soal3/web/index.html`) memakai rancangan tersebut dengan perbaikan bug, lalu diintegrasikan ke Supabase: login (created_by dari JWT), pilihan aktivitas arus kas, daftar jurnal POSTED + hash, tombol jurnal pembalik, audit log, dan tombol verifikasi hash.

Inti logika indikator seimbang (JavaScript):

```
function cents(v) { return Math.round((Number(v) || 0) * 100); }   // hindari galat desimal
const totalD = filled.reduce((s, l) => s + cents(l.debit), 0);
const totalC = filled.reduce((s, l) => s + cents(l.credit), 0);
const balanced = totalD > 0 && totalD === totalC;                  // zero-discrepancy
$('btnSubmit').disabled = !rules.every(r => r[1]);                 // semua aturan wajib ✓
```

Hasil uji otomatis (Playwright): Submit **terkunci** saat selisih Rp100.000 dan saat akun Kas didebit, **aktif** saat Debit = Kredit = Rp1.000.000, tidak ada error JavaScript, dan tampilan HP (390 px) tanpa scroll horizontal.

b. NotebookLM: verifikasi aturan bisnis terhadap PSAK

Log lengkap ada di `soal3/prompts/02_notebooklm_log.md`.

Sumber (6): artikel BINUS tentang PSAK 2 (Laporan Arus Kas) dan tentang setara kas & overdraft, ringkasan IAS 7 (IAS Plus/Deloitte), dokumen IAI tentang penomoran ulang PSAK (PSAK 2 → **PSAK 207**, berlaku 1 Januari 2024), spesifikasi aturan bisnis kode, dan file SQL backend.

Ringkasan verifikasi dari NotebookLM:

Aturan dalam kode	Status	Inti alasan
Definisi akun kas (Kas, Bank/giro, Kas Kecil)	Perlu perbaikan	Ketiga akun sudah tepat sebagai *kas*, tetapi **setara kas** (investasi sangat likuid, jatuh tempo ≤ 3 bulan **sejak tanggal perolehan**, risiko perubahan nilai tidak signifikan) dan **cerukan bank** belum dimodelkan.
Voucher kas keluar: kas hanya di kredit	Perlu perbaikan	Aturan ini benar untuk kas keluar. Namun **mutasi antar pos kas/setara kas** (Bank → Kas Kecil, Bank → deposito 3 bulan) **bukan arus kas** menurut PSAK 207, sehingga perlu jalur tersendiri. Klasifikasi aktivitas operasi/investasi/pendanaan juga belum ada.
Penguncian POSTED, storno, hash	Di luar cakupan PSAK 207, mendukung keandalan (PSAK 208)	Data arus kas tidak dapat dimanipulasi retroaktif. Koreksi dilakukan lewat jurnal pembalik yang saling meniadakan tanpa menghapus jejak audit.

Tindak lanjut (sudah diimplementasikan & diuji): akun `1104 Setara Kas (≤ 3 bulan)` dan `1105 Deposito > 3 bulan (bukan setara kas)`, tabel `cash_equivalent_accounts` sebagai satu sumber definisi, RPC `rpc_create_cash_transfer` (jenis `MUTASI_KAS`, dikecualikan dari arus kas), view `v_kas_dan_setara_kas` (saldo kredit Bank → cerukan, disajikan sebagai liabilitas jangka pendek), serta kolom `cash_flow_activity` + view `v_arus_kas_per_aktivitas`. Saran kode NotebookLM juga dikritisi: nilai `'ASSET_LANCAR'` yang disarankannya akan melanggar CHECK constraint, dan logika cerukan dibatasi hanya untuk akun Bank.

3.2 Mekanisme Jejak Audit (Audit Trail & Immutability)

Spesifikasi backend berupa **Supabase RPC** (fungsi PostgreSQL yang dipanggil `supabase.rpc()`). Kode: `soal3/supabase/01_audit_trail_immutability.sql`, hak akses: `02_supabase_security.sql`.

Spesifikasi fungsi

Fungsi / objek	Spesifikasi
`rpc_create_cash_voucher(tanggal, no_bukti, deskripsi, lines jsonb, aktivitas)`	Validasi ulang semua aturan bisnis, simpan header **DRAFT** + baris, lalu ubah ke **POSTED** (memicu segel hash). Mengembalikan id, created_by, created_at, hash.
`rpc_reverse_journal(id, alasan)`	**Storno**: hanya untuk jurnal POSTED, alasan wajib, maksimal satu kali per jurnal (`UNIQUE reversal_of`). Membuat jurnal baru `STORNO-<no>` dengan debit/kredit ditukar. Jurnal asli tetap utuh.
`rpc_verify_journal_integrity()`	Menghitung ulang SHA-256 tiap jurnal POSTED dan memeriksa rantai `prev_hash`.
Trigger `trg_journal_headers_immutable` / `trg_journal_lines_immutable`	**UPDATE / DELETE** pada jurnal POSTED (atau barisnya) → `RAISE EXCEPTION` yang menyarankan storno. Menambah baris ke jurnal POSTED juga ditolak.
Trigger `*_no_truncate`	`TRUNCATE` diblokir (TRUNCATE melewati trigger baris).
`audit_log` + `trg_audit_*`	Setiap INSERT/UPDATE/DELETE dicatat (actor, waktu, data lama/baru dalam JSONB). Tabel ini sendiri **append-only**.
RLS + `REVOKE` + `GRANT EXECUTE ... TO authenticated`	Klien tidak bisa menulis tabel langsung. Hanya pengguna login yang dapat memanggil RPC.

created_by, created_at, hash_checksum

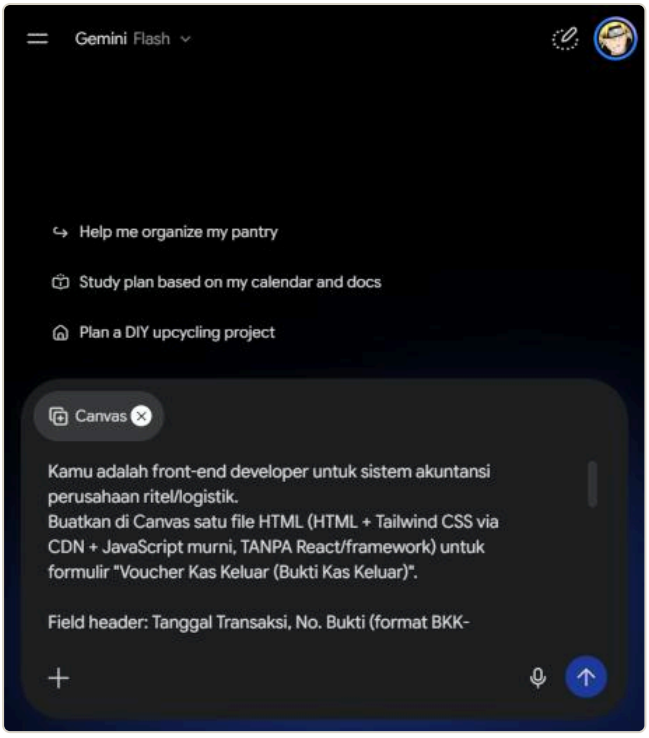
- `created_by` diisi oleh trigger dari klaim JWT Supabase (`current_setting('request.jwt.claims')->>'email'`), yaitu email pengguna yang login. Nilai yang dikirim klien **diabaikan** (uji 9: upaya memalsukan `direktur@palsu.com` tetap tercatat sebagai pengguna sebenarnya). Kolom ini juga tidak bisa diubah saat UPDATE.
- `created_at` = `now()` waktu server (bukan jam komputer pengguna), bertipe `TIMESTAMPTZ`. Ditambah `posted_by` dan `posted_at` saat posting.
- `hash_checksum` = `SHA-256( id | tanggal | no_bukti | deskripsi | jenis | created_by | created_at(UTC) | reversal_of | daftar akun:debit:kredit | prev_hash )`. Hash disegel saat status berubah menjadi POSTED. Karena memuat `prev_hash` (hash jurnal sebelumnya, urut `post_seq`), hash membentuk **rantai**: mengubah satu angka di jurnal mana pun akan terdeteksi. Proses posting diserialisasi dengan `pg_advisory_xact_lock` agar rantai tidak bercabang.

Hasil uji (`bukti/uji_soal3_audit_trail.txt`, PostgreSQL 16 + diterapkan di Supabase)

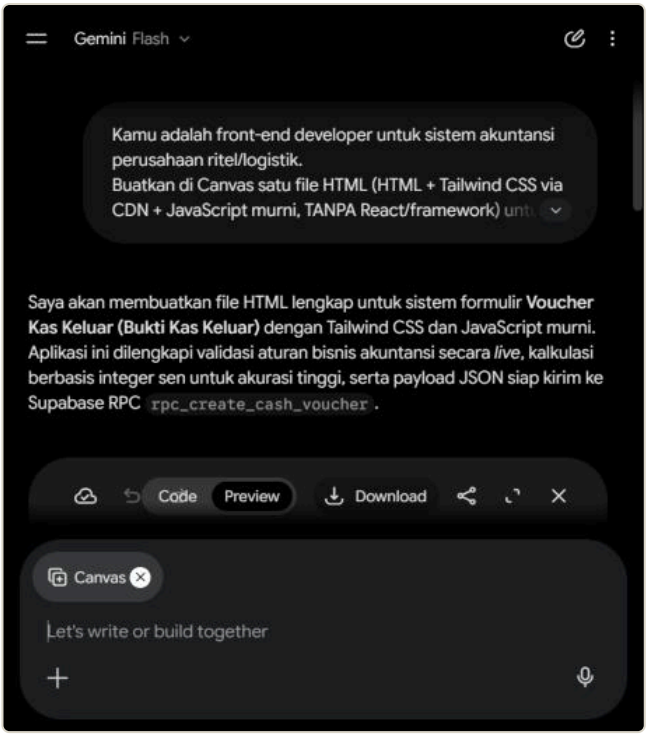
No	Skenario	Hasil
1	Voucher seimbang via RPC	✅ POSTED, created_by & hash terisi otomatis
2–4	Tidak seimbang / mendebit Kas / tanpa akun kas	❌ Ditolak RPC
5–7	UPDATE, DELETE, sisip baris pada jurnal POSTED	❌ Ditolak trigger
8	TRUNCATE	❌ Ditolak
9	Memalsukan created_by/created_at	✅ Diabaikan, diisi server
10–11	Storno, lalu storno kedua kali	✅ Berhasil / ❌ Ditolak
13	Verifikasi seluruh hash	✅ Semua valid
14	DBA mematikan trigger & mengubah nominal langsung	⚠️ **Terdeteksi**: `hash_valid = false`
15	Mengubah/menghapus audit_log	❌ Ditolak
Supabase	Klien tanpa login memanggil RPC / insert / select	❌ `permission denied` / data kosong

Dokumentasi tangkapan layar Soal 3

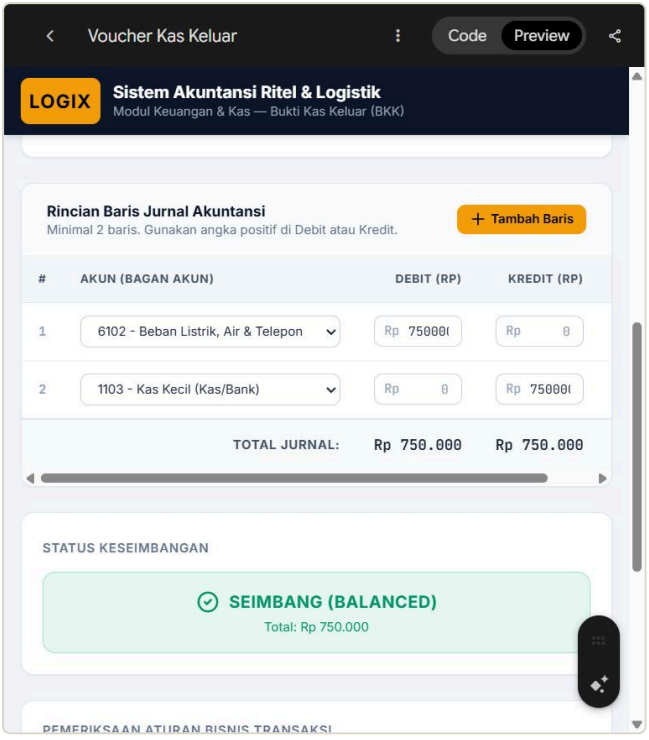
3.1a Gemini Canvas



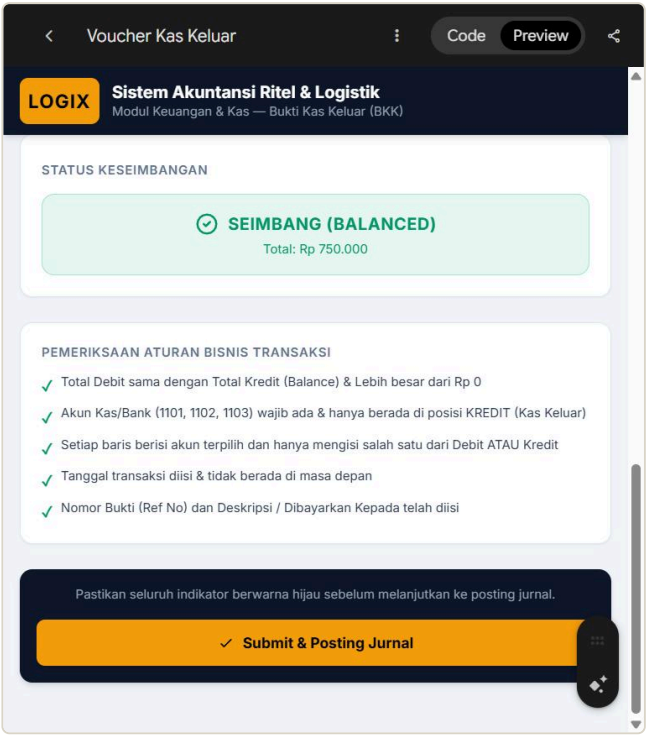
Prompt dikirim ke Gemini (mode Canvas)



Gemini membuat file HTML di Canvas

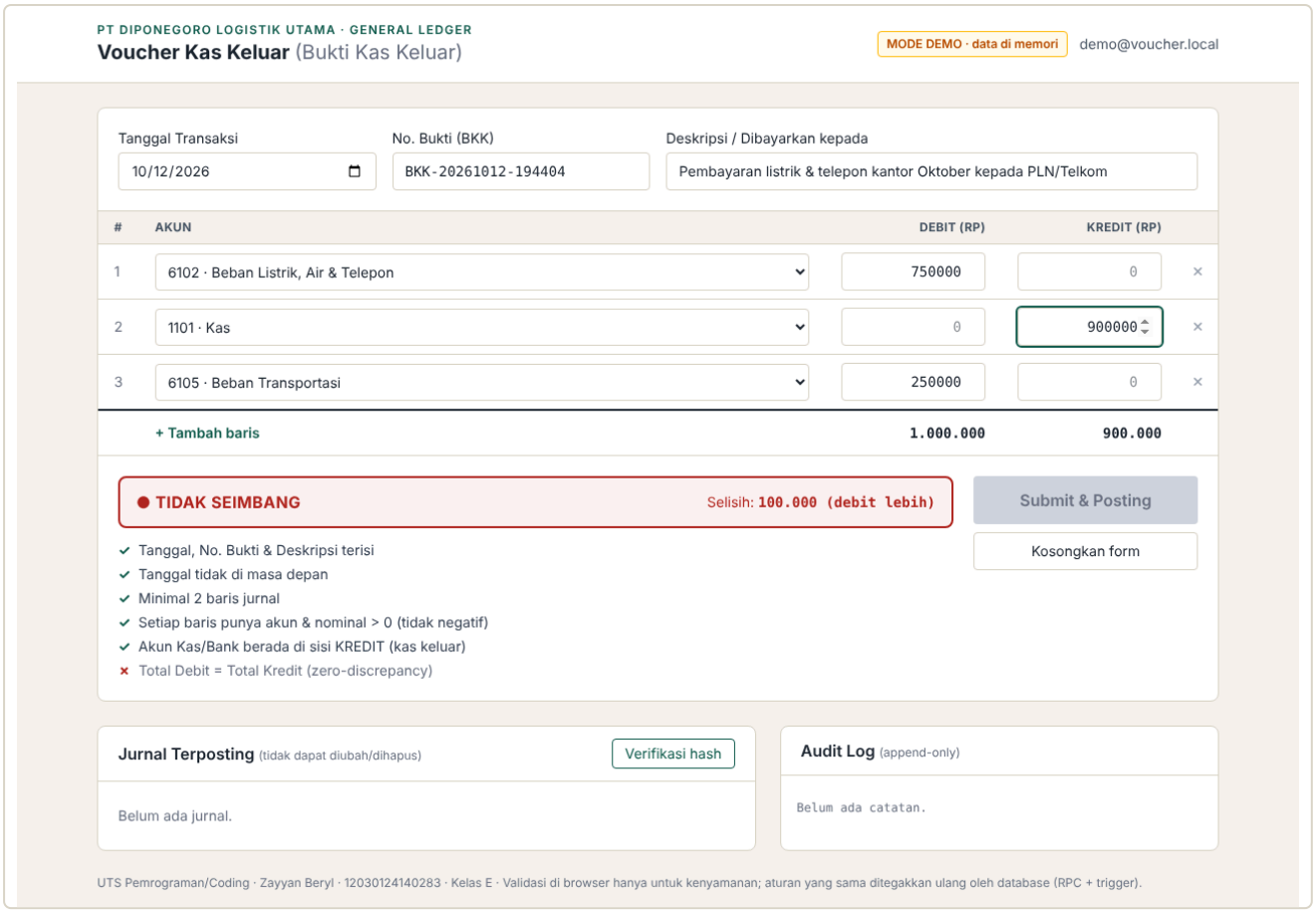


Preview: tabel jurnal dinamis + indikator SEIMBANG

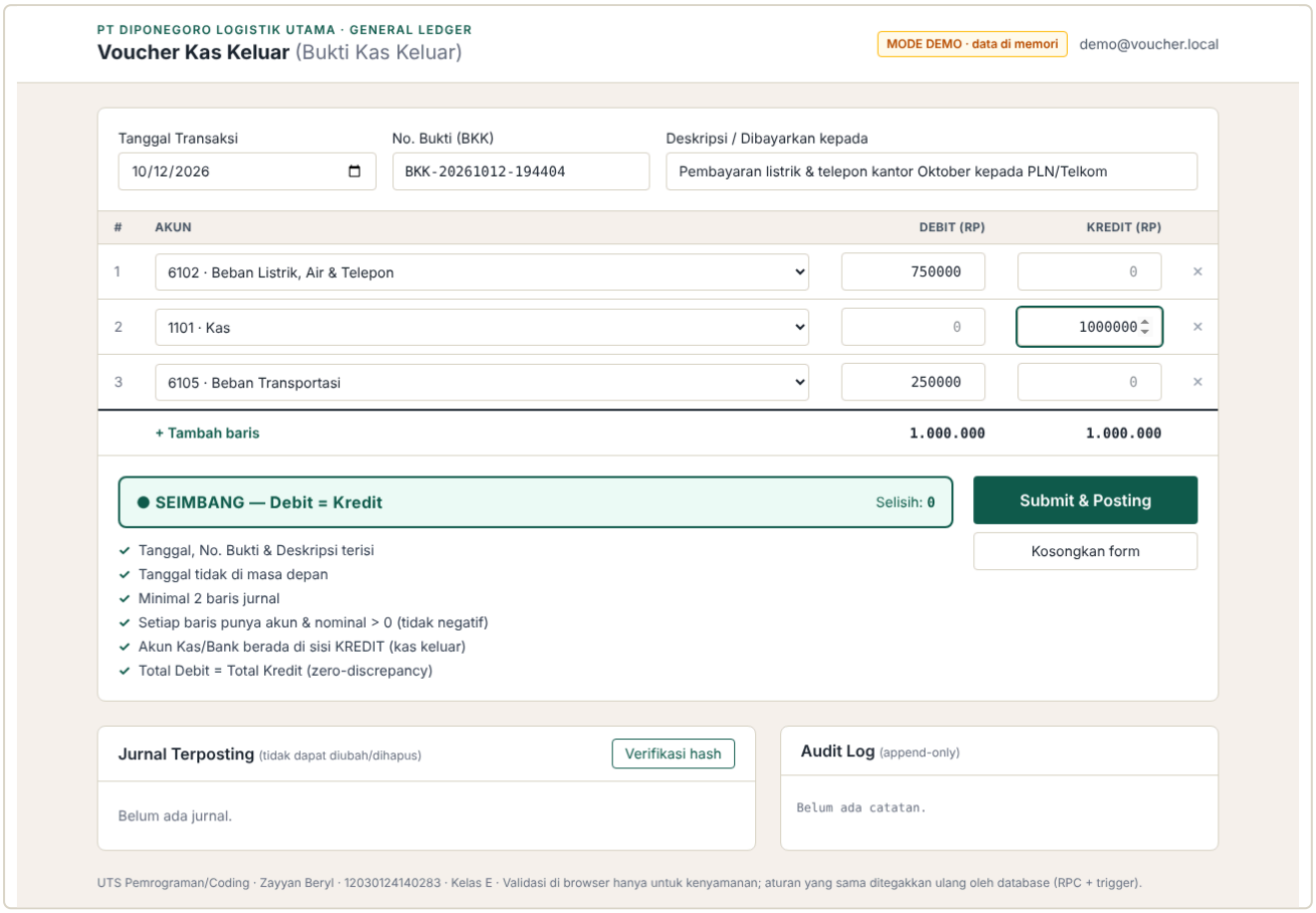


Preview: checklist aturan bisnis & tombol Submit

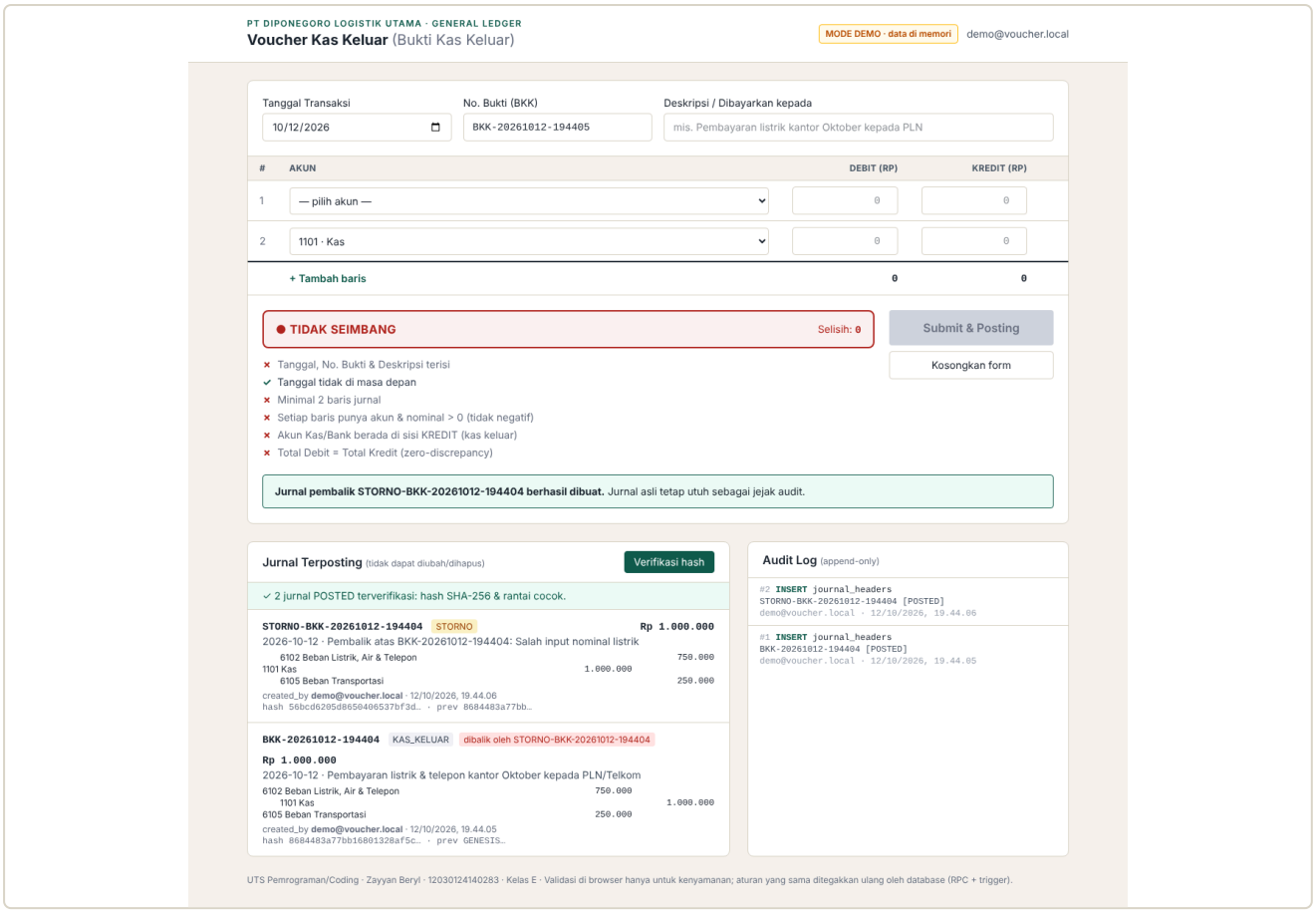
Versi final formulir (setelah code review & integrasi Supabase)



Tidak seimbang: indikator merah, selisih ditampilkan, Submit terkunci

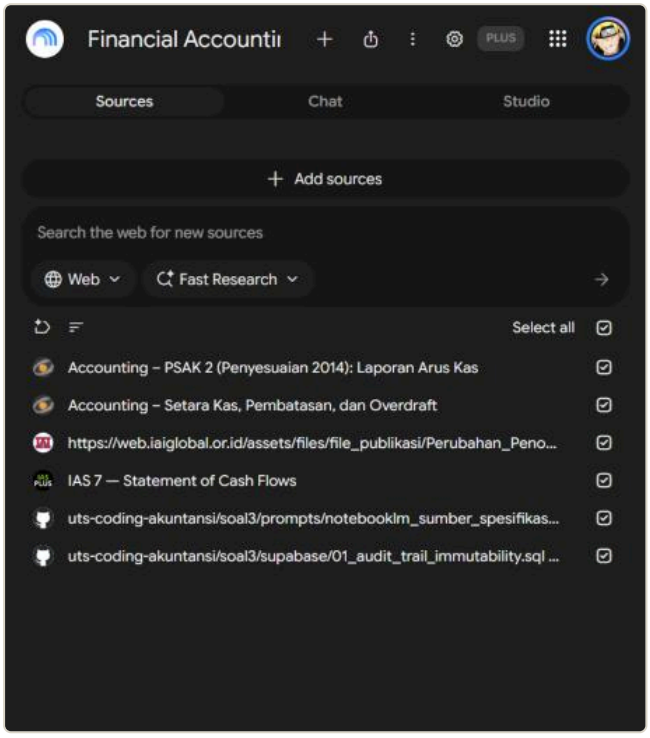


Seimbang: semua aturan ✓, Submit aktif

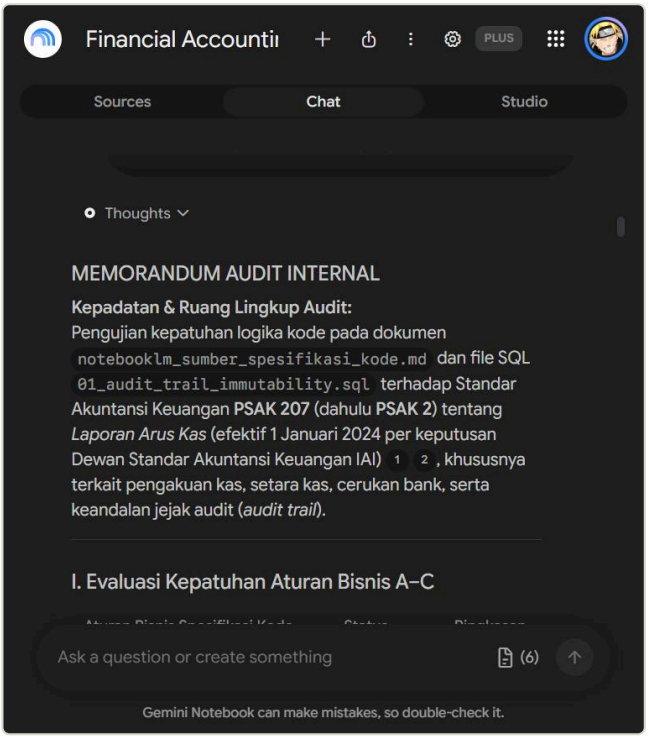


Setelah submit: jurnal POSTED dengan created_by & hash, storno, audit log, verifikasi hash

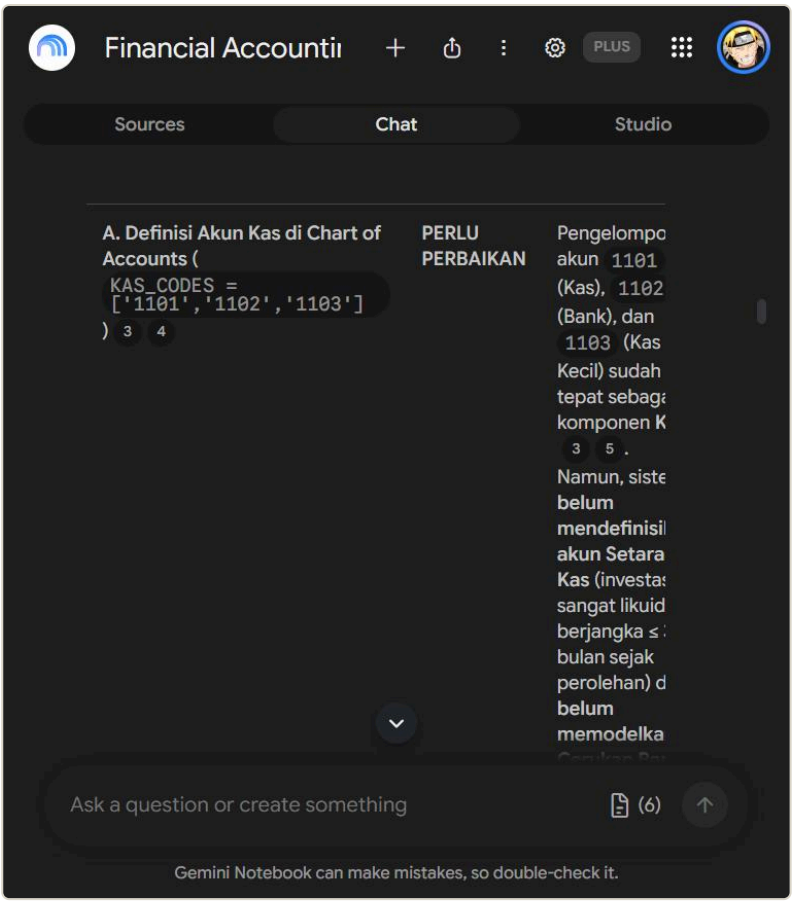
3.1b NotebookLM



Enam sumber: PSAK 2/207, setara kas, IAS 7, penomoran IAI, spesifikasi & kode



Jawaban NotebookLM: memo audit internal



Temuan A: setara kas & cerukan bank belum dimodelkan

Tindak lanjut temuan NotebookLM (bukti uji)

```
=== UJI 1: Mutasi internal Bank -> Kas Kecil lewat voucher KAS KELUAR -> DITOLAK (bukan arus kas keluar)
psql:soal3/supabase/05_uji_perbaikan_notebooklm.sql:10: ERROR:  Voucher kas KELUAR tidak boleh mendebit
akun kas/setara kas (Kas Kecil). Gunakan rpc_create_cash_transfer untuk mutasi internal.
=== UJI 2: Mutasi yang sama lewat rpc_create_cash_transfer -> BERHASIL (jenis MUTASI_KAS)
 mutasi
---------
 MK-0001
(1 row)

=== UJI 3: Penempatan deposito 3 bulan (setara kas) dari Bank -> BERHASIL sebagai MUTASI_KAS
 mutasi
---------
 MK-0002
(1 row)

=== UJI 4: Deposito > 3 bulan (1105) BUKAN setara kas -> mutasi DITOLAK
psql:soal3/supabase/05_uji_perbaikan_notebooklm.sql:19: ERROR:  Mutasi kas hanya antar akun kas/setara kas
(PSAK 207: bukan arus kas).
=== UJI 5: Voucher kas keluar operasi dengan aktivitas eksplisit
 aktivitas
-----------
 OPERASI
(1 row)

=== SALDO KAS & SETARA KAS (cerukan bank terdeteksi bila saldo Bank negatif)
 account_code |               account_name               |  category  |    saldo    |     penyajian
--------------+-------------------------------------------+------------+-------------+-------------------
-
 1101         | Kas                                       | KAS        |  5545000.00 | KAS_DAN_SETARA_KAS
 1102         | Bank                                      | KAS        | 13000000.00 | KAS_DAN_SETARA_KAS
 1103         | Kas Kecil                                 | KAS        |  2000000.00 | KAS_DAN_SETARA_KAS
 1104         | Setara Kas - Deposito Berjangka <= 3 Bulan | SETARA_KAS |  5000000.00 | KAS_DAN_SETARA_KAS
(4 rows)
```

```
=== ARUS KAS PER AKTIVITAS (MUTASI_KAS dikecualikan)
      aktivitas      | arus_kas_bersih
---------------------+-----------------
 OPERASI             |     -3000000.00
 PENDANAAN           |     30000000.00
 BELUM_DIKLASIFIKASI |     -1455000.00
(3 rows)

=== Integritas hash setelah perubahan skema
 valid | total
-------+-------
    13 |    13
(1 row)
```

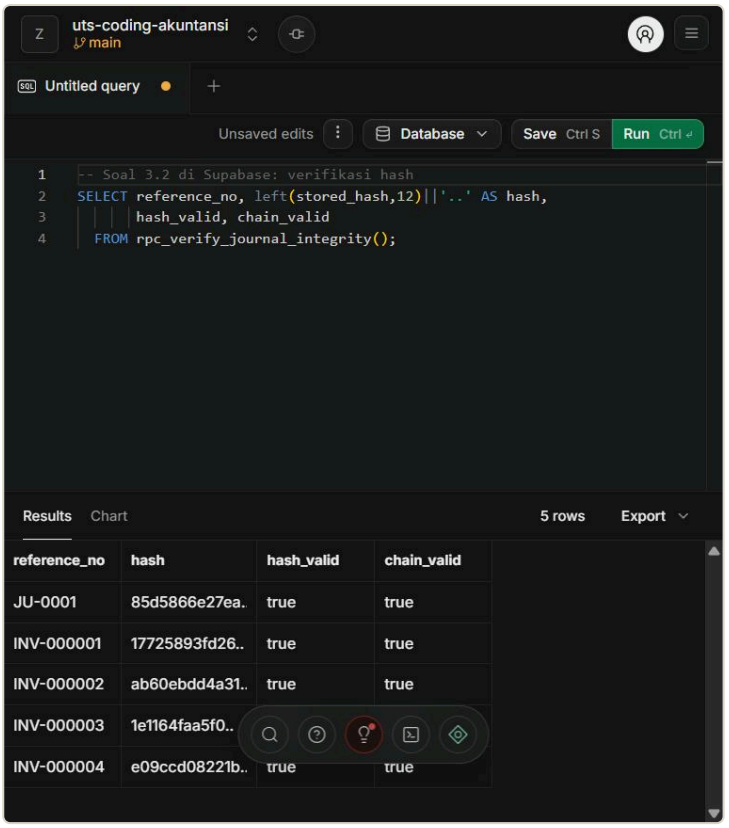
3.2 Bukti uji audit trail

```
=== UJI 5: UPDATE nominal / deskripsi jurnal POSTED -> DITOLAK
psql:soal3/supabase/03_uji_soal3.sql:31: ERROR:  Jurnal 10 (BKK-2026-10-001) sudah POSTED dan bersifat
final: UPDATE ditolak. Gunakan jurnal pembalik: SELECT rpc_reverse_journal(10, 'alasan');
psql:soal3/supabase/03_uji_soal3.sql:33: ERROR:  Baris jurnal milik jurnal POSTED tidak dapat di-UPDATE .
Gunakan jurnal pembalik (storno).
=== UJI 6: DELETE jurnal POSTED / barisnya -> DITOLAK
psql:soal3/supabase/03_uji_soal3.sql:36: ERROR:  Baris jurnal milik jurnal POSTED tidak dapat di-DELETE .
Gunakan jurnal pembalik (storno).
psql:soal3/supabase/03_uji_soal3.sql:37: ERROR:  Jurnal 10 (BKK-2026-10-001) sudah POSTED dan bersifat
final: DELETE ditolak. Gunakan jurnal pembalik: SELECT rpc_reverse_journal(10, 'alasan');
=== UJI 7: Menyisipkan baris baru ke jurnal POSTED -> DITOLAK
psql:soal3/supabase/03_uji_soal3.sql:41: ERROR:  Baris jurnal milik jurnal POSTED tidak dapat di-INSERT .
Gunakan jurnal pembalik (storno).
=== UJI 8: TRUNCATE tabel jurnal -> DITOLAK
psql:soal3/supabase/03_uji_soal3.sql:44: ERROR:  TRUNCATE pada tabel journal_lines dilarang (jejak audit).
```

```
=== UJI 13: Verifikasi integritas seluruh jurnal POSTED (hash & rantai)
      reference_no      |   hash_tersimpan    | hash_dihitung_ulang | hash_valid | chain_valid
------------------------+---------------------+---------------------+------------+-------------
 JU-0001                | e97106844d1a147d... | e97106844d1a147d... | t          | t
 INV-000001             | 03e90be808b07ca4... | 03e90be808b07ca4... | t          | t
 INV-000002             | 1b5fe56e59a6bd89... | 1b5fe56e59a6bd89... | t          | t
 INV-000003             | a54ca8344eeab8be... | a54ca8344eeab8be... | t          | t
 INV-000008             | 28067050b2668895... | 28067050b2668895... | t          | t
 INV-000009             | 87ce7131dd4ed3a5... | 87ce7131dd4ed3a5... | t          | t
 BKK-2026-10-001        | 03b15b0b47386112... | 03b15b0b47386112... | t          | t
 STORNO-BKK-2026-10-001 | fee725e839c52da1... | fee725e839c52da1... | t          | t
 BKK-2026-10-001R       | 4f76818bf47d7846... | 4f76818bf47d7846... | t          | t
(9 rows)

=== UJI 14: Simulasi peretas/DBA nakal mematikan trigger & mengubah nominal langsung
    -> hasil verifikasi setelah manipulasi:
 reference_no | hash_valid | chain_valid
--------------+------------+-------------
 JU-0001      | f          | t
(1 row)

    -> (ROLLBACK: data dikembalikan seperti semula)
```



Verifikasi hash dijalankan di Supabase SQL Editor (project cloud)

Uji keamanan Supabase dari browser (tanpa login)

```
1) supabase.rpc('rpc_create_cash_voucher', {...})          -> "permission denied for function
rpc_create_cash_voucher"
2) supabase.from('journal_headers').insert({...})            -> "permission denied for table
journal_headers"
3) supabase.from('journal_headers').select('reference_no')   -> [] (RLS aktif, tanpa policy baca -> tidak
ada data bocor)

Kesimpulan: walaupun publishable key terlihat di kode frontend, klien tanpa login tidak dapat
membaca maupun menulis jurnal. Pengguna yang login hanya bisa menulis lewat fungsi RPC
yang memvalidasi aturan bisnis; UPDATE/DELETE jurnal POSTED tetap ditolak trigger.
```

Lampiran: Struktur Repositori & Cara Menjalankan

Nama	Zayyan Beryl
NIM	12030124140283
Mata kuliah	Pengkodean dan Pemrograman — Kelas E
Dosen	Totok Dewayanto
Basis data	PostgreSQL 16 (uji lokal) & Supabase (cloud)

Repositori ini berisi seluruh artefak jawaban UTS: skema DDL, fungsi trigger SQL, kode backend (Supabase RPC) & frontend, log prompt AI, serta bukti pengujian. Jawaban naratif lengkap ada di berkas PDF jawaban.

Struktur

```
soal1/                          Desain data, normalisasi & asas berpasangan (25%)
  01_schema_general_ledger.sql    DDL chart_of_accounts, journal_headers, journal_lines
                                  + constraint trigger keseimbangan Debit = Kredit
  02_uji_soal1.sql                8 skenario uji
  erd.mmd / erd.png               ERD (Mermaid)
  JAWABAN_SOAL1.md

soal2/                          Trigger persediaan perpetual & concurrency (40%)
  01_inventory_trigger.sql        trg_update_stock_and_journal() + jurnal HPP/Persediaan otomatis
  02_uji_trigger.sql              6 skenario uji + rekonsiliasi GL vs stok fisik
  03_concurrency_snippets.sql     SELECT ... FOR UPDATE, SERIALIZABLE + retry, atomic update
  concurrency_demo.sh             simulasi 2 kasir bersamaan (stok 1 unit)
  JAWABAN_SOAL2.md

soal3/                          Purwarupa web full-stack & orkestrasi AI (35%)
  supabase/
    SETUP_LENGKAP_SUPABASE.sql    satu skrip untuk SQL Editor Supabase (soal1+2+3)
    01_audit_trail_immutability.sql  kunci POSTED, storno, created_by/created_at, hash SHA-256 berantai,
audit_log, RPC
    02_supabase_security.sql      RLS + REVOKE + GRANT RPC
    03_uji_soal3.sql              15 skenario uji
  web/index.html                  Formulir Voucher Kas Keluar (HTML + Tailwind + JS)
  prompts/                        log prompt Gemini Canvas & NotebookLM
  JAWABAN_SOAL3.md

bukti/                          output uji (txt) & tangkapan layar
run_all_tests.sh                bangun ulang DB & jalankan seluruh uji
```

Cara menjalankan

PostgreSQL lokal

```
createdb uts
DB=uts ./run_all_tests.sh      # hasil tersimpan di bukti/
```

Supabase

1. Buka *SQL Editor* → tempel isi `soal3/supabase/SETUP_LENGKAP_SUPABASE.sql` → *Run*.
2. *Authentication* → *Users* → *Add user* (centang *Auto Confirm User*).
3. Buka `soal3/web/index.html` di browser lalu login. Tambahkan `?demo` di URL untuk mencoba tanpa database (mode demo).

Ringkasan desain

- **Integritas di database, bukan di frontend.** CHECK, FK `ON DELETE RESTRICT`, constraint trigger keseimbangan (DEFERRED), dan aturan posting DRAFT → POSTED.
- **Persediaan perpetual.** Setiap mutasi stok langsung membuat jurnal Persediaan/HPP dalam transaksi yang sama, dan stok negatif ditolak.
- **Concurrency.** Baris item dikunci dengan `SELECT ... FOR UPDATE` di dalam trigger.
- **Jejak audit.** Jurnal POSTED immutable (UPDATE/DELETE/TRUNCATE ditolak), koreksi lewat storno, `created_by` diambil dari JWT di sisi server, `hash_checksum` SHA-256 berantai, dan `audit_log` bersifat append-only.